Kafka KRaft Controller Quorum DR — Case A: Complete Loss of DC1 in Confluent Platform EE 8.3.0

Executive Summary

The starting state under consideration is five KRaft voters: DC1 = C101, C102, C103 and DC2 = C201, C202. The majority is three. After a complete loss of DC1, only two of the five votes remain, so the old quorum {C101, C102, C103, C201, C202} can neither elect a leader nor commit new metadata entries. Simply changing `controller.quorum.bootstrap.servers` or restarting C201/C202 does not change the voter set — in dynamic KRaft, the bootstrap list is used only to discover the quorum, while actual membership is state replicated by KRaft itself. Apache Kafka distinguishes a dynamic quorum (`kraft.version >= 1`) from a static one (`kraft.version=0` or the feature not present).

Confluent Platform 8.3.x is built on Apache Kafka 4.3.x; CP 8.3.x was released on June 17, 2026. For dynamic quorum, Confluent published a quorum-loss recovery procedure in 2026: stop the surviving controllers, measure their metadata epoch and log end offset, pick a single seed using the "highest epoch" rule (and only break ties with "largest LEO"), run offline `kafka-metadata-recovery reconfig force-standalone` on it, bring it up as a single-node quorum, then clear the stale copy of `__cluster_metadata-0` on the remaining controllers, start them as observers, and promote them via `kafka-metadata-quorum add-controller`. `force-standalone` rewrites the voter set to a single seed at a higher epoch and is an irreversible operation.

This is the correct recovery model for Case A, but there is an important documentation caveat: the detailed public `reconfig force-standalone` procedure is currently published by Confluent in the Disaster Recovery documentation for Confluent for Kubernetes, not as a separate bare-metal runbook for CP EE 8.3.0. `kafka-metadata-recovery` itself also appears in the public Confluent Platform documentation, but in other workflows. Therefore, for RPM/TAR/VM installations of Confluent EE 8.3.0, you must confirm locally — before the operation — that the subcommand exists and verify its exact syntax, and treat `force-standalone` itself as an operation that requires Confluent Support; Confluent explicitly recommends Support for the manual path precisely because the rebuild is irreversible.

The most important consistency limitation is the fact that the lost DC1 held the 3/5 majority. In Raft, an entry could have been committed by exactly C101+C102+C103 and never reached either C201 or C202. Therefore, even a perfectly executed rebuild from the best seed in DC2 does not guarantee RPO=0 for metadata. Confluent states explicitly that when the lost region held the majority of voters, recovery may lose some previously committed metadata. Topic data on the broker volumes is a separate matter and may survive despite the loss of the corresponding control-plane entries.

This leads to an important conclusion: there is no procedure that can reconstruct, from C201 and C202 alone, metadata entries that neither of these two controllers ever held. Identical epoch/LEO values on C201 and C202 increase confidence in the shared copy, but they do not prove that no later committed prefix existed exclusively on the DC1 majority. The final metadata RPO in this scenario is therefore undetermined without a copy of C101/C102/C103 or an external source of truth. This conclusion follows directly from the 3+2 topology and from Confluent's warning about losing a region that held the majority.

Recovery must never be replaced with the `kafka-storage format --standalone` command on C201 or C202. `kafka-storage format --standalone` is meant for bootstrapping a new, empty dynamic quorum; Kafka deliberately does not auto-format empty metadata directories, because allowing a majority of controllers to start with an empty log could lead to electing a leader without any previously committed data. On an existing cluster, formatting a surviving seed would be a destructive creation of new bootstrap state, not a reconstruction of the old log.

After rebuilding C201+C202 we get a 2-voter quorum whose majority is still two: losing either of these two controllers will stop the quorum again. Therefore the 2-voter state must be treated as temporary. Operationally, at least a third new controller — referred to in this report as `<C203>` — should be provisioned as soon as possible, giving three voters and a majority of two. C203's location, hostname, `node.id`, port, `metadata.log.dir`, and infrastructure are unspecified in the question. The procedure for adding a fresh controller to an existing dynamic quorum uses `kafka-storage ... --no-initial-controllers`, followed by synchronization as an observer, and `kafka-metadata-quorum add-controller`.

The entire recovery can be performed without writing any custom Java code. It requires only the tools shipped with Kafka/Confluent, a shell, configuration, and possibly the system's process stop/start mechanism.

Assumptions, Scope, and Input Conditions

This runbook assumes Confluent Platform Enterprise 8.3.0, and therefore the Kafka 4.3.x family. The public platform/current documents describe the 8.3.x family and may include fixes from later patches; therefore, standard Kafka 4.3 elements such as dynamic membership, `kafka-metadata-quorum`, `kafka-storage`, and KRaft v1 are well-defined at this line's level, while the availability of the exact vendor implementation of `kafka-metadata-recovery reconfig force-standalone` in the specific 8.3.0 artifact must be verified locally before performing the operation.

Item	State in Case A
Original voter set	C101, C102, C103, C201, C202
Majority	3
Survivors after the disaster	C201, C202
Normal election possible in the old quorum	No — 2 < 3
DC1 disks/backups	None, per the assumption
C201/C202 metadata	Local directories are assumed to have survived; their actual integrity must be verified
node.id for C201/C202	Unspecified; "C201" does not automatically mean node.id=201
Controller hostnames/ports	Unspecified; examples use <C201_CTRL> and <C202_CTRL>
metadata.log.dir	Unspecified
TLS/SASL	Unspecified
systemd unit / service name	Unspecified
Broker topology and broker survivability	Unspecified
Static vs. dynamic quorum	Unspecified and critical
External metadata inventory / IaC	Unspecified

Item	State in Case A
Target post-DR topology	Unspecified beyond C201/C202

The first decision gate is therefore the type of quorum. Confluent and Apache define a dynamic quorum as `kraft.version >= 1`, with `controller.quorum.bootstrap.servers` and without `controller.quorum.voters`; a static quorum has `kraft.version=0` or lacks the feature, and configures all voters via `controller.quorum.voters`. The standard `add-controller` and `remove-controller` commands apply to a dynamic quorum.

On a healthy cluster, proper verification looks as follows:

```
$CONFLUENT_HOME/bin/kafka-features.sh \
  --bootstrap-controller <CONTROLLER_HOST:CONTROLLER_PORT> \
  describe
```

`FinalizedVersionLevel: 1` for `kraft.version` means a dynamic quorum; `0` or the feature being absent means static. After a quorum loss, this command may of course fail to return a response, so you also need to check the preserved configuration and prior operational data.

On C201 and C202, at minimum check:

```
grep -E \
'^(process.roles|node.id|metadata.log.dir|log.dirs|listeners|controller.listener.names|cont
/etc/kafka/controller.properties
```

The path `/etc/kafka/controller.properties` is only an example; Confluent installs sample `broker.properties`, `controller.properties`, and `server.properties` files under `/etc/kafka`, but the path used in a given deployment remains unspecified. A controller-only production configuration uses `process.roles=controller`, a unique `node.id`, a controller listener, and `controller.listener.names`.

If the configuration and preserved information confirm a static quorum, this manual `force-standalone → Observer → add-controller` runbook must be stopped. Public dynamic-membership procedures do not allow you to simply edit `controller.quorum.voters` on C201/C202 and treat them as a new 2-voter quorum. A static→dynamic migration is normally performed on a running cluster by upgrading `kraft.version`, and Confluent's prerequisites for that migration require healthy brokers and controllers. In a 2/5 state, that condition is not met. Any further path for a static quorum must be pursued with Confluent Support.

If the quorum is dynamic, setting `controller.quorum.bootstrap.servers=C201,C202` will not by itself shrink the voter set. This is merely a list of addresses used to discover the current quorum. Do not attempt a "fix" that consists only of removing C101–C103 from this parameter.

KRaft Mechanics During and After Quorum Loss

In this scenario, several layers that are often confused in practice must be kept separate: the physical metadata log, the Raft state and High Watermark, the MetadataImage, the KafkaRaftClient, and the active QuorumController.

Layer	What happens in Case A
Metadata log `__cluster_metadata-0`	C201 and C202 retain their own local copies of the log/snapshots, but may have different end points
Raft membership	Still logically 5 voters, until offline recovery creates a new voter set
Leader	After losing C101–C103, it is impossible to obtain 3 votes, so the old quorum cannot elect a leader
High Watermark	Marks the boundary of the committed log known to a given member; may be unknown (-1) or stuck at the last known committed point
LEO	The local end of the log; may be larger than HW, because the tail of the log may be uncommitted
Metadata Image	Built from a snapshot and committed records delivered by Raft; must not be equated with "the whole LEO"
KafkaRaftClient	Responsible for Raft roles, election, replication, HW, and delivering committed records to listeners
QuorumController	The active controller performs control-plane operations only after acquiring the active-controller role, which follows from Raft leadership

Confluent exposes precise `kafka.server:type=raft-metrics` metrics: `current-state` can take values such as `leader`, `candidate`, `follower`, `unattached`, `observer`; `current-leader=-1` means an unknown leader, `high-watermark=-1` means an unknown HW, and `log-end-offset` shows the current end of the local Raft log.

LEO is not a commit point. In the Kafka 4.3 code, a `KafkaRaftClient` follower sets its own HW to the minimum of its own end offset and the HW advertised by the leader. The leader updates its own HW via `LeaderState`, writes it to the log, and only then notifies listeners of the newly committed range. This is exactly why you must never manually "set" the HW or arbitrarily copy segments from C201 to C202 or vice versa.

The election that follows a rebuild also matters. In Kafka 4.3, after a Candidate→Leader transition, `KafkaRaftClient` initializes a new epoch and immediately writes a start-of-epoch control record. A comment in the code explains that the High Watermark can only advance after the election once a record originating from the new leader's epoch has been written. Therefore, after `force-standalone`, you should not expect the process to simply "read the old HW value" — a new, legitimate Raft sequence begins in the new epoch.

`KafkaRaftClient` only delivers the committed range to listeners: when catching a listener up, it reads the log with `Isolation.COMMITTED`, bounded by the High Watermark. If an older range is needed, it may first deliver a snapshot and then committed records from the log.

This directly determines how the Metadata Image is rebuilt. `MetadataLoader` waits until the High Watermark is known, and then until the loaded offset reaches HW-1; only then does it finish the initial catch-up phase and initialize/publish the metadata publishers. When it receives a snapshot, it builds a `MetadataDelta`, replays the records it contains, applies the delta to the image, resets the batch loader to the resulting image, and only then publishes the result. For subsequent committed batches, `handleCommit` loads further records and updates the image.

`MetadataImage` in Kafka 4.3 includes, among other things, `FeaturesImage`, `ClusterImage`, `TopicsImage`, `ConfigurationsImage`, client quotas, producer IDs, ACLs, SCRAM credentials, and delegation tokens. Consequently, a lost committed fragment of the metadata log can mean a logical absence of, for example, a topic definition, an assignment, a configuration, an ACL, or a feature update — even though

the corresponding topic data may still physically exist on the broker disks. This follows both from the scope of MetadataImage and from Confluent's explicit distinction between a possible loss of metadata and preservation of topic data.

`QuorumController` does not itself carry out the Raft election. It is the `KafkaRaftClient`/Raft state machine that determines leadership; the active `QuorumController` can then generate control-plane records. In the Kafka 4.3 code, a write event is rejected if the local `QuorumController` is not the active controller, and when it is active, the result of the operation is passed to `raftClient.prepareAppend`. On the commit path, the active controller completes operations waiting on committed/stable offsets, while standbys replay committed metadata to keep their own state up to date.

This leads to the single most important DR rule: **do not choose the seed based on the largest LEO.** Confluent requires first choosing the largest metadata epoch, and using LEO only as a tie-breaker. A longer log with a lower epoch may belong to an old, diverged branch, and choosing it can irrecoverably lose committed metadata.

Recovery Options and Path Selection

Option	Consistency	RTO	Requirements	Assessment for Case A
Offline rebuild from the best of C201/C202 via force-standalone	Preserves the best available DC2 copy, but does not guarantee all previous commits, since the lost DC1 held the majority	Shortest realistic option	Dynamic KRaft, an intact seed, the recovery tool, Support for the manual bare-metal workflow	Recommended path assuming a permanent loss of DC1
Recovering even a single original DC1 controller from its storage	Best possibility of recovering the full committed prefix and a normal 3/5 election	Potentially longer	A physical disk/node from DC1	Logically the safest option, but ruled out by the "no DC1 backups/nodes" assumption
CFK `kubectl confluent kraft recover-region`	Same seed-recovery model; automates park/snapshot/rebuild/rejoin	Shorter and better checkpointed	CFK and a matching plugin version	Preferred by Confluent only when the deployment is CFK
Editing `controller.quorum.bootstrap.servers` to C201,C202	Does not reconstruct the voter set	No solution	—	Not a recovery method; bootstrap servers are only discovery hints
`kafka-storage format --standalone` on C201/C202	May destroy the only surviving metadata copy	Deceptively fast	—	Forbidden as a DR method; `format` is for bootstrapping new storage, not recovering existing storage

Option	Consistency	RTO	Requirements	Assessment for Case A
A new cluster plus manual metadata re-creation	New cluster identity; requires re-creating many categories of metadata	Very high	Full external inventory and a data-migration plan	Ultimate last resort — not a "reconstruction of the old quorum"

Given the user's assumptions, the correct path is therefore:

> C201/C202 offline → DC2 snapshots → compare epoch/LEO → pick the best seed → force-standalone → seed becomes single-node leader → C202 with a clean `__cluster_metadata-0` as Observer → full catch-up → add-controller → provision a new third voter as soon as possible → validate metadata against an external source of truth.

This is exactly the model published by Confluent for quorum-loss recovery.

Detailed Recovery Runbook

The commands below use the wrapper names found in current Confluent distributions (`bin/kafka-metadata-quorum`, `bin/kafka-storage`, `bin/kafka-metadata-shell`). Apache documentation often shows the equivalent names with a `.sh` suffix. The exact `$CONFLUENT_HOME`, properties path, ports, security properties, and service name are unspecified.

Let's assume the following environment variables:

```
export CONFLUENT_HOME=/opt/confluent

# Do NOT assume that the name "C201" means node.id=201.
export C201_CTRL='<c201-fqdn>:<controller-port>'
export C202_CTRL='<c202-fqdn>:<controller-port>'

# Local root directory containing meta.properties and __cluster_metadata-0.
export MDIR='<metadata.log.dir>'

# Client/security properties used by the admin tools.
export ADMIN_CFG='<path-to-kafka-client.properties>'

# Server properties for each controller.
export CONTROLLER_CFG='<path-to-controller.properties>'

# The systemd unit name is environment-specific and was not provided.
export KAFKA_UNIT='<KAFKA_CONTROLLER_UNIT>'
```

Step A — Freeze the situation and forbid starting the lost C101–C103

You must not allow an accidental restart of the old voter set during recovery. Stop C201 and C202 using a controlled process-management mechanism. In its own procedure, Confluent first "parks" all surviving controllers, so that no KRaft process is using the metadata log during offline inspection/reconfiguration.

For systemd:

```
sudo systemctl stop "$KAFKA_UNIT"

sudo systemctl is-active "$KAFKA_UNIT"
pgrep -af 'kafka.Kafka|KafkaRaft' || true
```

`$KAFKA_UNIT` is unspecified; do not mechanically assume the name `confluent-kafka`.

Step B — Confirm cluster and controller identity

On both surviving hosts:

```
grep -E '^(cluster.id|node.id|directory.id)=' \
  "$MDIR/meta.properties"

grep -E \
'^(process.roles|node.id|metadata.log.dir|log.dirs|listeners|controller.listener.names|cont
  "$CONTROLLER_CFG"
```

`cluster.id` on C201 and C202 must be identical; `node.id` must match the corresponding configuration and must be unique. Do not change `cluster.id`, `node.id`, or the seed's existing `directory.id` during recovery. Controllers added later are formatted using the same cluster ID. Apache requires that a single cluster ID be preserved when formatting all members of a given cluster.

If C201 and C202 show different `cluster.id` values, STOP: at least one copy does not belong to the same cluster, or there is a serious operational inconsistency.

Step C — Confirm a dynamic quorum

If prior operational documentation is available, check the earlier result:

```
$CONFLUENT_HOME/bin/kafka-features.sh \
  --bootstrap-controller "$C201_CTRL" \
  describe
```

During the current outage, this command may time out. `kraft.version=1` or higher indicates dynamic; `0`/absent indicates static. An additional signal is the absence of `controller.quorum.voters` together with the presence of `controller.quorum.bootstrap.servers`.

If you cannot reliably confirm a dynamic quorum, do not perform `force-standalone` per this runbook. The type of quorum is unspecified in the assumptions and constitutes a hard safety gate.

Step D — Take fresh, local backups of DC2

The prohibition on using DC1 backups does not mean you should perform irreversible operations without a snapshot of C201/C202. Before recovery, Confluent requires a snapshot of every surviving KRaft volume and describes it as a reliable rollback point, especially in case the wrong seed is chosen.

A storage/VM/LVM/SAN-level snapshot is preferred; its exact command is unspecified, since no platform was given. In addition, while Kafka is stopped, you can also take a file-level copy on a different filesystem:

```
export SNAPDIR='<separate-safe-filesystem>/kraft-dr-$(date -u +%Y%m%dT%H%M%SZ)'
mkdir -p "$SNAPDIR"

tar --numeric-owner --acls --xattrs \
  -C "$MDIR" \
  -cpf "$SNAPDIR/kraft-metadata.tar" .

sha256sum "$SNAPDIR/kraft-metadata.tar" \
  > "$SNAPDIR/kraft-metadata.tar.sha256"

sha256sum -c "$SNAPDIR/kraft-metadata.tar.sha256"
```

Do this separately on C201 and C202, and label the artifacts with the hostname.

Step E — Sanity-check the physical metadata log

Apache and Confluent provide `kafka-dump-log` to decode KRaft segments, and `kafka-metadata-shell` to browse the metadata structure.

Example:

```
ls -lah "$MDIR/__cluster_metadata-0"

LATEST_LOG=$(
  find "$MDIR/__cluster_metadata-0" -maxdepth 1 -type f -name '*.log' \
  | sort | tail -1
)

echo "$LATEST_LOG"

$CONFLUENT_HOME/bin/kafka-dump-log \
  --cluster-metadata-decoder \
  --files "$LATEST_LOG" \
  > "/tmp/$(hostname)-latest-kraft-segment.txt"
```

If a snapshot exists:

```
LATEST_SNAPSHOT=$(
  find "$MDIR/__cluster_metadata-0" -maxdepth 1 \
    -type f -name '*.checkpoint' \
  | sort | tail -1
)

if [ -n "$LATEST_SNAPSHOT" ]; then
  $CONFLUENT_HOME/bin/kafka-dump-log \
    --cluster-metadata-decoder \
    --files "$LATEST_SNAPSHOT" \
    > "/tmp/$(hostname)-latest-kraft-snapshot.txt"
fi
```

```
Apache documents exactly this decoding for both `.log` files and `<offset>-<epoch>.checkpoi
```

Interactive inspection:

```
$CONFLUENT_HOME/bin/kafka-metadata-shell \
  --directory "$MDIR/__cluster_metadata-0"
```

Example commands inside the shell:

```
ls /
ls /topics
```

Apache shows that the metadata shell exposes, among other things, `brokers`, `metadataQuorum`, `topicIds`, and `topics`.

A corrupt segment, CRC errors, a missing `meta.properties`, a mismatched `cluster.id`, or a completely missing `__cluster_metadata-0` on one of the two hosts is grounds to stop and investigate before choosing a seed.

Step F — Verify the recovery utility before use

On CP EE 8.3.0:

```
test -x "$CONFLUENT_HOME/bin/kafka-metadata-recovery" \
  && echo "kafka-metadata-recovery present"

$CONFLUENT_HOME/bin/kafka-metadata-recovery --help

$CONFLUENT_HOME/bin/kafka-metadata-recovery \
  reconfig log-length --help

$CONFLUENT_HOME/bin/kafka-metadata-recovery \
  reconfig force-standalone --help
```

If `reconfig force-standalone` does not exist in the exact 8.3.0 build you installed, STOP. Do not attempt to replicate its behavior using `kafka-storage format`, manual snapshot editing, or custom Java code. Confluent's official manual recovery documentation defines `kafka-metadata-recovery reconfig force-standalone` specifically as an irreversible voter-set rebuild, and recommends performing the manual variant with Support.

Step G — Offline measurement of epoch and LEO on both surviving controllers

Kafka must be stopped. Confluent uses the following pattern, including the `.lock` file:

On C201:

```
[ -f "$MDIR/.lock" ] || : > "$MDIR/.lock"

$CONFLUENT_HOME/bin/kafka-metadata-recovery \
  reconfig log-length \
  --metadata-log-dir "$MDIR"
```

On C202, exactly the same:

```
[ -f "$MDIR/.lock" ] || : > "$MDIR/.lock"

$CONFLUENT_HOME/bin/kafka-metadata-recovery \
  reconfig log-length \
  --metadata-log-dir "$MDIR"
```

Record the results in the DR log:

```
C201: epoch = <E201>, LEO = <L201>
C202: epoch = <E202>, LEO = <L202>
```

The selection rule is absolute:

```
if E201 > E202  => seed = C201
if E202 > E201  => seed = C202
if E201 = E202  => seed = the controller with the larger LEO
if epoch and LEO are equal => both are equally valid candidates based on these two criteria
```

Do not choose based on LEO before epoch. This is one of the most important corrections relative to the intuitive procedure; Confluent warns against permanently losing metadata by picking a longer but older branch.

The remaining examples assume C201 as the seed. If C202 wins instead, swap the names symmetrically.

Step H — Final gate before the irreversible operation

Before running `force-standalone`, everything below must be true:

```
[OK] Kafka stopped on C201 and C202
[OK] Cluster ID matches on both copies
[OK] Dynamic KRaft confirmed
[OK] Recovery tool and force-standalone verified via local --help
[OK] Snapshot of C201 and C202 taken and verified
[OK] Epoch/LEO recorded for both controllers
[OK] Seed chosen epoch-first, LEO-second
[OK] No plan to start C101/C102/C103 with the old metadata
[OK] The risk of losing committed metadata from the DC1 majority is understood
```

If any of these points is not true, do not proceed. Confluent explicitly identifies seed selection and the voter-set rebuild as irreversible parts of recovery.

Step I — force-standalone only once, only on the C201 seed

The official manual Confluent procedure has the following form:

```
[ -f "$MDIR/.lock" ] || : > "$MDIR/.lock"

$CONFLUENT_HOME/bin/kafka-metadata-recovery \
  reconfig force-standalone \
  --config "$ADMIN_CFG"
```

In CFK, `$ADMIN_CFG` is a Kafka client-properties file mounted by the operator. The exact content/path of the corresponding file on bare-metal EE 8.3.0, including TLS/SASL, is unspecified; you must use the syntax verified via the local `--help` output and the documentation/support for the exact package you installed. Do not arbitrarily substitute `server.properties` for it if your local version of the tool does not expect that.

The operation rewrites the voter set so the seed becomes the sole voter, and does so at a higher epoch.

After success, do not run `force-standalone` a second time.

If the command hangs, returns an ambiguous status, or is interrupted by a signal/reboot, do not repeat it. Confluent warns that it may have partially rewritten the voter set, and re-running it can corrupt the metadata state.

Step J — Ensure the seed has the correct dynamic discovery configuration

For a dynamic quorum, the seed's configuration should keep its original `node.id` and listener, and use `controller.quorum.bootstrap.servers` rather than the static `controller.quorum.voters`. Confluent recommends the dynamic form:

```
process.roles=controller
node.id=<EXISTING_NODE_ID_C201>

listeners=CONTROLLER://<c201-fqdn>:<controller-port>
controller.listener.names=CONTROLLER

controller.quorum.bootstrap.servers=<c201-fqdn>:<controller-port>,<c202-fqdn>:<controller-p

# MUST be absent for a dynamic quorum:
# controller.quorum.voters=...
```

The bootstrap list may also include controllers that are not yet reachable, but after DR it is worth eventually updating it to reflect actually existing endpoints. Simply changing this list does not change voter membership.

`controller.quorum.auto.join.enable` is unspecified in the input. Confluent Platform has supported auto-join for new dynamic controllers since 8.2, when set to `true`; therefore, before manually running `add-controller`, always check `CurrentVoters` first, so you don't try to add a controller that has already been auto-promoted.

Step K — Start C201 and elect the single-node leader

```
sudo systemctl start "$KAFKA_UNIT"
sudo systemctl status "$KAFKA_UNIT" --no-pager
```

Then:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --status
```

Expected state:

```
ClusterId:       <same as before DR>
LeaderId:        <NODE_ID_C201>
LeaderEpoch:     <new epoch>
HighWatermark:   <value >= 0 once stabilized>
CurrentVoters:   [C201]
CurrentObservers: [...]
```

Confluent describes exactly this effect: after `force-standalone`, the seed starts up and becomes the leader of a single-node quorum. The standard `describe --status` shows `ClusterId`, `LeaderId`, `LeaderEpoch`, `HighWatermark`, `CurrentVoters`, and `CurrentObservers`.

There is no universal expected numeric HW value — it is unspecified, since it depends on the seed's log. Once stabilized, however, it should be known, not -1. `HW <= LEO` must always hold; on a fully caught-up single-node quorum, HW can reach LEO once new records from the leader's epoch have been written. The mechanics of the Kafka code require an entry from the new epoch before HW can advance.

Also check:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --replication
```

and the feature level:

```
$CONFLUENT_HOME/bin/kafka-features.sh \
  --bootstrap-controller "$C201_CTRL" \
  --command-config "$ADMIN_CFG" \
  describe
```

Expect `kraft.version >= 1`.

Step L — Verify that MetadataLoader has finished reconstruction

In C201's logs, look for the exact classes/events:

```
sudo journalctl -u "$KAFKA_UNIT" --since '<RECOVERY_START_ISO>' \
  | grep -Ei \
  'KafkaRaftClient|MetadataLoader|QuorumController|high.?watermark|leader|voter|snapshot|fa
```

The log path is unspecified if the deployment does not use journald.

During initialization, `KafkaRaftClient` logs the reading of the KRaft snapshot/log and the initial voter set. `MetadataLoader` logs, among other things, the completion of catch-up to the current High Watermark and the loading of the snapshot; detailed HW changes inside `KafkaRaftClient` are logged at debug level.

For go-live on the seed, you should observe the following logical sequence:

```
KRaft log/snapshot read
        ↓
C201 elected Leader
        ↓
HW becomes known / starts advancing
        ↓
MetadataLoader loads snapshot + committed batches up to HW
        ↓
Metadata Image is published
        ↓
QuorumController can act as the active controller
```

This matters more than the mere fact that the Java process shows status RUNNING.

Step M — Prepare C202 for rejoin

C202 must not be started with its old, pre-DR branch of `__cluster_metadata-0` and simply "join" as a voter. Confluent's official procedure removes the stale metadata partition on the non-seed node, starts the controller empty, lets it fetch state from the seed as an Observer, and only then adds it as a voter.

Since we kept a snapshot, instead of an irrecoverable `rm`, you can additionally move the directory to a filesystem outside the active `$MDIR`:

```
# Kafka on C202 is still stopped.
export HOLD='<separate-safe-filesystem>/c202-pre-rejoin'
mkdir -p "$HOLD"

mv "$MDIR/__cluster_metadata-0" \
   "$HOLD/__cluster_metadata-0"
```

This implements logically the same "clear stale metadata" step, while preserving a forensic artifact. Confluent's documentation for a recovering region explicitly talks about moving the stale metadata partition aside, and then starting an empty controller as an Observer.

Do not run `kafka-storage format --standalone` on C202.

Check C202's configuration:

```
process.roles=controller
node.id=<EXISTING_NODE_ID_C202>

listeners=CONTROLLER://<c202-fqdn>:<controller-port>
controller.listener.names=CONTROLLER

controller.quorum.bootstrap.servers=<c201-fqdn>:<controller-port>,<c202-fqdn>:<controller-p

# dynamic quorum:
# no controller.quorum.voters
```

Dynamic controllers use `controller.quorum.bootstrap.servers`; the list does not need to be an exact mapping of the voter set.

Step N — Start C202 as an Observer and let it fully catch up

```
sudo systemctl start "$KAFKA_UNIT"
```

On C201:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --status
```

Expect C202 to appear under `CurrentObservers`, if it was not auto-joined:

```
CurrentVoters:
  C201

CurrentObservers:
  ...
  C202
```

Then:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --replication
```

Apache and Confluent both require starting the new controller, monitoring `describe --replication`, and only running `add-controller` once it has caught up to the active controller.

Promotion condition:

```
C202 LogEndOffset == leader LogEndOffset
C202 Lag == 0
C202 is regularly fetching from the leader
C202 is not reporting snapshot/CRC/storage errors
```

Do not promote a controller that is still lagging.

Step O — Add C202 as a voter

First, check `CurrentVoters` again. If C202 is already a voter — for example, via `controller.quorum.auto.join.enable=true` — skip this command.

In the manual workflow, Confluent runs `add-controller` from the process/environment of the non-seed controller:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
```

```
  --bootstrap-controller "$C201_CTRL" \
  add-controller
```

Apache Kafka 4.3 and Confluent Platform use the same command shape for the controller endpoint.

Verify immediately:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --status
```

Expected stage:

```
CurrentVoters = [C201, C202]
LeaderId      = C201 or, after a later election, any current voter
majority      = 2
tolerance for losing voters = 0
```

Do not treat this as the target HA state.

Step P — Add a new third controller

`<C203>` is a symbolic name; its host, `node.id`, listener, directory, and placement are unspecified.

Example controller configuration:

```
process.roles=controller
node.id=<NEW_UNIQUE_NODE_ID>

listeners=CONTROLLER://<c203-fqdn>:<controller-port>
controller.listener.names=CONTROLLER

controller.quorum.bootstrap.servers=<c201-fqdn>:<controller-port>,<c202-fqdn>:<controller-p
```

Read the preserved cluster ID, for example from C201:

```
CLUSTER_ID=$(
  awk -F= '$1 == "cluster.id" {print $2}' \
  "$MDIR/meta.properties"
)

printf '%s\n' "$CLUSTER_ID"
```

A fresh controller being added to an existing dynamic cluster is formatted using `--no-initial-controllers`:

```
$CONFLUENT_HOME/bin/kafka-storage \
  format \
  --cluster-id "$CLUSTER_ID" \
  --config /etc/kafka/controller-c203.properties \
  --no-initial-controllers
```

Start C203, then:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --replication
```

Once `Lag=0`:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  add-controller
```

Again, if `controller.quorum.auto.join.enable=true` and C203 has already appeared as a voter, skip `add-controller`. Confluent has supported auto-join since CP 8.2 as an alternative way of building a dynamic voter set.

After the third voter:

```
CurrentVoters = [C201, C202, C203]
majority = 2
tolerance = 1 controller
```

Only this state restores basic tolerance for a single controller failure.

Step Q — Update bootstrap hints on brokers and controllers

For a dynamic quorum, Confluent requires that all nodes use `controller.quorum.bootstrap.servers` instead of `controller.quorum.voters`. The list does not need to contain exactly all voters, but it should make it possible to discover the quorum.

Target state:

```
controller.quorum.bootstrap.servers=<c201>:9093,<c202>:9093,<c203>:9093
```

Port 9093 is just an example; the actual port is unspecified.

If the broker configs still contain:

```
controller.quorum.bootstrap.servers=C101:...,C102:...,C103:...,C201:...,C202:...
```

and C201/C202 are reachable, the mere presence of dead bootstrap hints does not, by itself, mean the old voter membership is still in effect. Even so, they should eventually be replaced with live endpoints, and if brokers need to be restarted, do a rolling restart only after the controllers have stabilized. For dynamic configuration changes, Confluent recommends stabilizing the controllers first, then restarting brokers one at a time.

Step R — Do not later restore the old C101/C102/C103 with stale metadata

If DC1 is ever rebuilt, no old `__cluster_metadata-0` carrying the pre-DR voter set may simply be started alongside the recovered quorum. Confluent requires that such metadata first be moved aside, that the controller be started empty as an Observer, synchronized with the recovered leader, and only later added as a voter.

In Case A, per the assumption there are no old DC1 copies, so the new DC1 nodes should be treated as fresh controllers joining the existing cluster ID, not as a source of old metadata.

State Transitions and Component Interaction

The diagram below shows the logical change in quorum state. The key points — rebuilding down to a single seed, Observer status before promotion, and re-adding voters — correspond to Confluent's official workflow.

Diagram 1 — Quorum state transitions

```
Healthy5 [5 voters, majority=3]
   |  loss of C101, C102, C103
   ▼
QuorumLost [C201 + C202 = 2/5, no majority, no legitimate leader]
   |  stop C201 and C202
   ▼
Frozen
   |  snapshot + epoch/LEO
   ▼
SeedSelected
   |  force-standalone on the best seed
   ▼
Reconfigured
   |  start the seed
   ▼
SingleLeader [CurrentVoters = [seed], new epoch,
              KafkaRaftClient elects a leader, HW becomes known,
              MetadataLoader rebuilds the image]
   |  C202 starts without the stale metadata
   ▼
ObserverSync
   |  Lag=0 + add-controller
   ▼
TwoVoters [majority = 2, zero tolerance for losing a voter]
   |  provision C203 + sync + add
   ▼
ThreeVoters [majority = 2, tolerates one failure]
```

It is worth noting how the meaning of the High Watermark changes between stages. Before DR, C201/C202 may hold a local HW resulting from old commits; after the seed is newly elected within the single-voter set, `KafkaRaftClient` creates a new leader epoch and writes a start-of-epoch control record, enabling the HW to advance further. Listeners only receive data below the HW, via `Isolation.COMMITTED`.

The internal sequence looks as follows (participants: Operator, C201 seed, KafkaRaftClient, MetadataLoader, QuorumController, C202, Brokers):

Diagram 2 — Internal recovery sequence

```
 1. Operator      → C201 seed        : stop KRaft
 2. Operator      → C201 seed        : snapshot metadata
 3. Operator      → C201 seed / C202 : log-length -> epoch, LEO
 4. Operator      → C201 seed        : force-standalone
 5. Operator      → C201 seed        : start
 6. C201 seed     → KafkaRaftClient  : initialize snapshot + raft log
 7. KafkaRaftClient                  : Candidate -> Leader
 8. KafkaRaftClient                  : append start-of-epoch control record
 9. KafkaRaftClient                  : establish/advance High Watermark
10. KafkaRaftClient → MetadataLoader : snapshot + committed records up to HW
11. MetadataLoader                   : rebuild MetadataImage
12. MetadataLoader → QuorumController: published committed metadata
13. QuorumController                 : active controller state available
14. Operator      → C202             : move stale __cluster_metadata-0 aside
15. Operator      → C202             : start controller
16. C202          → C201 seed        : discover recovered quorum
17. C201 seed     → C202             : snapshot / committed raft records
18. C202                             : Observer catches up
19. Operator      → C201 seed        : add-controller C202
20. KafkaRaftClient                  : commit voter-set change (voters = C201, C202)
21. Brokers       → C201 seed        : reconnect / register / control-plane RPCs
22. C201 seed     → Brokers          : recovered MetadataImage / control decisions
```

The Kafka 4.3 code confirms that, during initialization, `KafkaRaftClient` reads the KRaft snapshot and log, and then determines the current voter state. After acquiring leadership, it writes a start-of-epoch record before the HW can make further progress.

`MetadataLoader`, in turn, does not publish the initial image until the High Watermark is known and the loader has reached HW-1; the snapshot is replayed into a `MetadataDelta`, and the result becomes the new `MetadataImage`.

In practice, the expected operational states are as follows:

Stage	Voter set	C201	C202	Leader	Metadata Image
Before the outage	C101, C102, C103, C201, C202	voter	voter	one of the 5, unspecified	normal
After losing DC1	still the old 5	no quorum	no quorum	no legitimate new majority	local state may exist, but the control plane has no healthy leader
After offline force-standalone	on-disk recovery voter set = seed	stopped	stopped	none to start	not yet published after restart
After C201 starts	C201	leader	stopped	C201	snapshot + committed log up to the new HW
After clean C202 starts	C201	leader	observer	C201	C202 rebuilds its copy from the leader
After add-controller	C201, C202	voter	voter	currently C201, may change later	synchronized
After C203	C201, C202, C203	voter	voter	any voter that gets elected	HA copies on 3 controllers

Do not expect `QuorumController` to "reconstruct missing entries" by analyzing broker data. Its operations are based on the reconstructed metadata state and Raft records; the logical content of `MetadataImage` comes from snapshots and committed metadata records.

Failure Modes, Rollback, Consistency Checks, and Final Verification

The most important rollback principle is distinguishing the state before and after `force-standalone`. Before the voter-set rewrite, you can go back to the local copies of C201/C202, but you only recover the old 2/5 state, still without a quorum. After a successful `force-standalone`, the correct operational strategy is essentially to roll forward on the same seed. Confluent explicitly warns against switching seeds midway through recovery, and never to repeat an uncertain `force-standalone`.

Failure	Action	Rollback / further path
`log-length` fails	Fix access/lock/storage and re-run	Confluent describes `log-length` as idempotent
C201/C202 have a different cluster ID	STOP	Do not merge or force. Determine the correct storage
Seed has a lower epoch but a larger LEO	Do not choose it	Choose the higher epoch
Cannot confirm dynamic KRaft	STOP	Support; do not use the dynamic add-controller workflow
`force-standalone` does not exist in the CP 8.3.0 binary	STOP	Support; do not substitute `kafka-storage format`
`force-standalone` failed/interrupted	Do not repeat	Preserve both snapshots, the logs, and the outcome; contact Confluent Support
`force-standalone` succeeded, C201 will not start	Do not automatically switch to C202	Keep C201 as the authoritative seed after the rewrite; diagnose storage/config/security
C201 starts, LeaderId=-1	Do not proceed	Check the voter rewrite, listener, node ID, logs, feature level
C201 is leader, but HW=-1 long after stabilization	Do not restore full traffic	Investigate the Raft state/log; -1 means an unknown HW
C202 does not appear as an Observer	Do not run add-controller	Check bootstrap, listener, TLS/SASL, cluster ID, and whether the stale metadata was actually moved aside
C202 has Lag > 0	Wait/fix replication	Apache requires catch-up before promotion
`add-controller` returns an ambiguous result	First run `describe --status`	If C202 is already in `CurrentVoters`, do not repeat unnecessarily. Confluent considers all remaining steps other than `force-standalone` safe to resume
A topic/config/ACL is missing after DR	Pause administrative mutations and start reconciliation	DC1-only committed records that never existed on C201/C202 cannot be recovered from them

Raft/quorum verification

The basic go/no-go check is:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
```

```
  --bootstrap-controller "$C201_CTRL" \
  describe --status
```

For the target three-voter quorum, expect:

```
ClusterId:       exactly the original cluster ID
LeaderId:        one of the current voters
LeaderEpoch:     positive / after recovery, higher than the previous recovery epoch
HighWatermark:   known, not -1
MaxFollowerLag:  0 once stabilized
CurrentVoters:   C201, C202, C203
```

The exact format of the modern output, including `directoryId` and controller endpoints, is documented by Apache Kafka 4.3.

Then:

```
$CONFLUENT_HOME/bin/kafka-metadata-quorum \
  --command-config "$ADMIN_CFG" \
  --bootstrap-controller "$C201_CTRL" \
  describe --replication
```

Once stabilized, every voter should be caught up; `Lag=0` is required in particular before promoting C202/C203.

Feature-level verification

```
$CONFLUENT_HOME/bin/kafka-features.sh \
  --bootstrap-controller "$C201_CTRL" \
  --command-config "$ADMIN_CFG" \
  describe
```

On the dynamic path:

```
Feature: kraft.version ... FinalizedVersionLevel: 1
```

or a higher level in future versions; for CP/Kafka 4.3, the public documentation describes a dynamic quorum at level 1.

JMX verification

On every controller, at minimum monitor the following MBeans/attributes:

```
kafka.server:type=raft-metrics
  current-state
  current-leader
  current-epoch
  high-watermark
  log-end-offset
  commit-latency-avg
  commit-latency-max
```

```
kafka.server:type=MetadataLoader,name=CurrentMetadataVersion
kafka.server:type=MetadataLoader,name=HandleLoadSnapshotCount

kafka.server:type=SnapshotEmitter,name=LatestSnapshotGeneratedBytes
kafka.server:type=SnapshotEmitter,name=LatestSnapshotGeneratedAgeMs
```

Confluent publishes exactly these metrics for KRaft. The JMX transport, exporter, port, and the way of reading them in this environment are unspecified.

After recovery, C201 should progressively show `current-state=leader`; C202 should show `observer` during sync, and later the `voter` role, depending on the current Raft state, once promoted; `current-leader` should be known, and `high-watermark` should not remain at -1.

Metadata Image verification via logs

Look for:

```
sudo journalctl -u "$KAFKA_UNIT" --since '<RECOVERY_START_ISO>' \
  | grep -E \
  'Reading KRaft snapshot and log|Starting voters|MetadataLoader|handleLoadSnapshot|finishe
```

In the Kafka 4.3 source, `KafkaRaftClient` logs the initial reading of the KRaft snapshot/log and the voter set, and `MetadataLoader` logs the completion of the initial catch-up to the HW as well as `handleLoadSnapshot`.

Logical metadata verification via the broker endpoint

Once the broker control plane is working again, it is worth exporting an inventory:

```
export BROKER='<surviving-broker>:<broker-port>'
```

Topics:

```
$CONFLUENT_HOME/bin/kafka-topics \
  --bootstrap-server "$BROKER" \
  --command-config "$ADMIN_CFG" \
  --list \
  | sort \
  > /tmp/post-dr-topics.txt
```

Partition description:

```
$CONFLUENT_HOME/bin/kafka-topics \
  --bootstrap-server "$BROKER" \
  --command-config "$ADMIN_CFG" \
  --describe \
  > /tmp/post-dr-topic-describe.txt
```

ACLs:

```
$CONFLUENT_HOME/bin/kafka-acls \
  --bootstrap-server "$BROKER" \
  --command-config "$ADMIN_CFG" \
  --list \
  > /tmp/post-dr-acls.txt
```

Topic configurations:

```
$CONFLUENT_HOME/bin/kafka-configs \
  --bootstrap-server "$BROKER" \
  --command-config "$ADMIN_CFG" \
  --entity-type topics \
  --describe \
  > /tmp/post-dr-topic-configs.txt
```

This check is especially important, because `MetadataImage` covers exactly the topic/cluster/config/ACL/SCRAM/quotas categories and others, and some committed records may have existed exclusively in the lost DC1 majority.

The results must be compared against an external source of truth: Terraform/Ansible, a CMDB, GitOps, an application-level topic catalog, ACL exports, prior monitoring, or another inventory. Whether such a source exists and how complete it is are unspecified in the question. Without DC1 and without an external baseline, it is not possible to prove — cryptographically or logically — that the last globally committed metadata offset equals the one held by the chosen seed.

Checking broker data must be treated separately from checking metadata. Confluent notes that after DR, broker-side topic data may remain intact, and brokers may be able to reconnect their log volumes even though some control-plane metadata may have been lost. So, if after recovery you find a broker directory holding segments for a topic that no longer appears in `MetadataImage`, do not "fix" the situation by manually adding its segments to the metadata log. A separate analysis of the topic ID, assignments, configuration, and external inventory is required.

Control Center should not be used as a tool to reconstruct the voter set. In Confluent's official quorum-loss workflow, corrective operations are performed via `kafka-metadata-recovery`, `kafka-metadata-quorum`, or the CFK plugin; Control Center may be used after the control plane is recovered to observe cluster state, but the public DR procedure does not define within it any operation equivalent to `force-standalone`.

Final GO criteria

All of the following should hold simultaneously:

```
Cluster ID = original
dynamic kraft.version confirmed
exactly one current leader
HighWatermark known
HW <= LEO on all controllers
voter lag = 0
at least 3 voters, if the C203 infrastructure is available
MetadataLoader has finished catch-up
no FATAL / corruption / repeated election loop
topics/configs/ACLs match the available external inventory
```

```
brokers are registering with the recovered controller
critical partitions are available
pre-DR snapshots of C201/C202 are still preserved
```

The C201/C202 snapshots must not be deleted right after the first green `describe --status`. Confluent recommends deleting recovery artifacts only after confirming the health of the full quorum.

Persistent Risk

The most serious lasting risk in this Case A remains the impossibility of proving RPO=0. Because DC1 held the 3/5 majority and was lost without backups, any committed metadata record that was accepted by C101+C102+C103 but not yet replicated to C201/C202 is, by definition, unavailable for recovery. Confluent documents exactly this problem for the case of losing a region that held the majority.

Looking ahead to future DR planning, the topology of "3 voters in DC1 + 2 in DC2" on its own cannot tolerate a complete loss of DC1, because losing DC1 removes the majority at the same time. With five voters, an arrangement across three failure domains — e.g., 2+2+1 — allows at least three votes to be retained after losing any single domain; however, the specific future architecture, the number of data centers, and latency requirements are outside the assumptions given here.

Official Sources Key to This Runbook

- Confluent Platform — Supported Versions and Interoperability — CP 8.3.x ↔ Kafka 4.3.x.
- Confluent Platform — Configure and Monitor KRaft — dynamic/static quorum, configuration, `kafka-metadata-quorum`, monitoring, metadata tools.
- Apache Kafka 4.3 — KRaft — provisioning, dynamic controller membership, `add-controller`, `remove-controller`, metadata quorum/debug tools.
- Confluent — Recover a Multi-Region KRaft Cluster from Quorum Loss — the official seed-selection algorithm, `log-length`, `force-standalone`, Observer→Voter, rollback warnings, and the case of losing a region that held the majority.
- Confluent — `kubectl confluent kraft recover-region` — describes the semantics of `force-standalone`: rewriting the voter set to a single seed at a higher epoch, and rejoining the non-seed nodes afterward.
- Apache Kafka 4.3 source — `KafkaRaftClient`, `MetadataLoader`, `QuorumController`, and `MetadataImage` — HW behavior, committed reads, MetadataImage reconstruction, and the relationship between Raft and the active controller.